

パイプライン仕様(①～⑦)

目次

1 ① リポジトリ解析	4
2 ② 関数仕様書	4
3 ③ テスト仕様書	4
4 ④ テストコード	4
5 ⑤ 実装	4
6 ⑥ テスト	5
7 ⑦ 完了検証	5
8 ⑧ 分析・改善	5

最初のトリガは必ず人が引く。AI が勝手に次フェーズへ進むことはない。起動の入口は3つあるが、実行の実体(headless Claude 1 回 + ファイル状態での契約検証)は 共通で、同じ実行枠ロックを取り合う。

表 1

フェーズ	起動 (チャット)	起動(ブラウザ)	入力	出力	完了判定(機械)
① 解析	/legacy-0-analyze	—	legacy/ 全部	functions.json ・ 骨子 ・ WBS ・ 規約	抽出器の完全性突合
① 仕様書	/legacy-1-spec F-xxxx	仕様書ページ / 連続実行	legacy/ 該当関数 +DK	docs/specs/*.md	status: reviewed(人 OK)
② テスト仕様	/legacy-2-testspec F-xxxx	仕様書ページ / 連続実行	①(reviewed) 規約	docs/test-specs/*.md	status: approved かつ spec-hash 一致
③ テストコード	/legacy-3-testcode F-xxxx	仕様書ページ / 連続実行	②(approved) 規約	tests/	ledger の freeze ハッシュ一致
④ 実装	/legacy-4-impl F-xxxx	仕様書ページ / 連続実行	①(reviewed) 規約	src/	モジュール存在かつスタブ検知ゼロ
⑤ テスト	/legacy-5-test F-xxxx	仕様書ページ / 連続実行	実行結果 +①②	docs/test-results/	result: pass(実装率 100% ・ 失敗 0)
⑥ 完了検証	/legacy-6-check	WBS トップ(全⑤完了後)	全成果物	completion-check.md	ledger check + review all が exit 0
⑦ 分析・改善	/legacy-7-analyze	WBS トップ(⑥pass 後)	src/ ・ docs/ ・ 計測	analysis.md ・ 施策票	テスト全 pass 維持(挙動保存)

連続実行は①～⑤を工程横断で自動的に回す(1 件ごとに全関数を再走査し、次に着手できる工程を選ぶ)。承認待ち・裁定待ちはスキップするので、自動で進む範囲と人が要る範囲がそのまま分かれる。詳細は画面設計。

情報遮断表

各フェーズが読んでよい入力の正(references/workflow.md より)。「読むはいけない」に触れたくなったら、それは仕様の穴－ISSUE を起票して停止する。

表 2

フェーズ	読んでよい入力	読むはいけないもの
① 解析	legacy/ 全部	—
① 仕様書	legacy/ 該当関数、functions.json、domain-knowledge.md	tests/、src/
② テスト仕様	①(reviewed)、conventions.md、domain-knowledge.md	legacy/ 、src/、tests/
③ テストコード	②(approved)、conventions.md	legacy/ 、①、src/
④ 実装	①(reviewed)、conventions.md	legacy/ 、②、 tests/
⑤ テスト	結果+①②(トリアージ判断用)、src/((a)修正時)	legacy/、tests/ の編集
⑦ 分析	src/・docs/・計測結果(全体を見る)	tests/ の編集(挙動保存が大原則)

レガシー原文を読む役割は ① と ①(改訂含む)だけ。

1 ① リポジトリ解析

- ・人が答えること: レガシーの場所・言語・新パッケージ名。型対応表・規約を一緒に確定
- ・Fortran の関数列挙は静的解析プログラム(extract_fortran.py)が行う。LLM の仕事は説明の充填と抽出レポート(data/extract-report.json)の確認のみ
- ・抽出器は固定・自由形式、継続行、COMMON/USE/intent、call+関数参照による呼出推定に対応
- ・再実行は常にマージ: func_id 不変・手修正保持・新規のみ追加。途中でやり直しても採番のやり直しや成果物の宙吊りは起きない
- ・完全性突合(2 系統カウンターの照合)が NG のファイルは監査ログに残り、調査対象になる
- ・出力: functions.json(正データ)・仕様書骨子・WBS・conventions.md・docs-sphinx/ テンプレ配置

2 ① 関数仕様書

- ・執筆単位は関数。仕様の各項目に Confidence(● 確認済 / ● 推測 / ● 仮定)と レガシー行番号の根拠を必須付与
- ・status 遷移: skeleton → draft → reviewed(人が OK して確定)
- ・draft 化の直後に機械レビュー(review_checks.py spec)を通し、NG ゼロにしてから人に出す
- ・● ● 項目は ISSUE(仮説+Yes/No)として人へ。回答は domain-knowledge.md に蓄積され再質問を防ぐ
- ・バッチモード: 「まとめて N 件」で複数関数を draft まで連続処理し、一斉レビュー表(docs/spec-review.md)で人がまとめて処理する。表は「承認できる → ISSUE あり → AI 修正待ち」の順に並び、行内のボタンで承認・修正依頼が完結する(画面設計)
- ・全件(数百~2000 件)は無人ドライバ pipeline.py で回す(運用設計参照)。ブラウザの連続実行でも同じことができ、そちらは①以降の工程も続けて進む
- ・draft の書き直しは自由。reviewed の書き直しは②が stale になるため人の了承を先にする
- ・spec-gap 改訂: ④が詰まって起票された ISSUE への対応も①の担当(レガシー確認・根拠付きで仕様を更新 → ハッシュ伝搬で下流に再生成が波及)

3 ② テスト仕様書

- ・入力は ①(reviewed)+規約+DK のみ。レガシーは読まない
- ・● の仕様項目すべてにテストケースを対応付ける(トレーサビリティ)。ケース ID(TC-xxx)と SPEC-ID の対応が機械レビューで突合される
- ・レガシー実行環境なしが基本前提のため、期待値は「仕様 ● +人間確認」で確定(実測は環境がある場合のみ)
- ・フロントマター spec-hash に生成時点の①ハッシュを記録(連鎖の起点)
- ・status 遷移: generated → approved(人の承認ゲート)。△未確定ゼロが承認条件
- ・承認依頼前に review_checks.py testspec を通し NG ゼロにする

4 ③ テストコード

- ・入力は ②(approved)+規約のみ。①もレガシーも読まない
- ・②の全ケースを pytest 関数に実装し、TC マーカーで対応付ける(pytest --collect-only+マーカー突合で過不足を機械検知)
- ・完了時に ledger freeze-tests でテストファイルのハッシュを台帳に記録。以降の改変は「△改変」として機械検知される
- ・④⑤中の tests/ 編集は hook が物理的に拒否。テスト側の疑義は ISSUE → 人承認 → ②③再生成が正規経路

5 ④ 実装

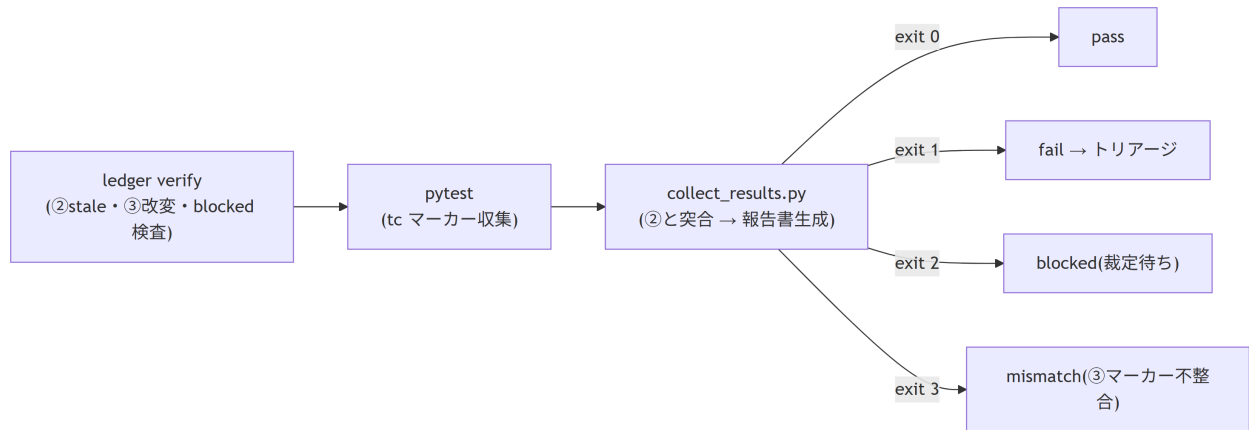
- ・入力は ①(reviewed)+規約のみ。②③もレガシーも読まない
- ・スタブ量産の禁止: 空実装・NotImplementedError・TODO/FIXME は check_stubs.py が検出し未完了扱い

- ・実装しきれない場合は spec-gap ISSUE を起票して停止 → ①改訂 → ハッシュ伝搬 → ④再開。自力でレガシーを見に行くことは構造上できない(遮断+hook)
- ・詳細仕様は docstring に書き、Sphinx で API ページ化(トレース対応表で仕様書と相互リンク)

6 ⑤ テスト

実行フロー(run_tests は⑤の一括 MCP ツール):

図 6-1



- ・pass 条件は「**実装率 100%**(②の全ケースが③に実装済み)かつ**失敗 0**」
- ・結果報告書は {func-id}_{YYYYMMDD-HHMMSS}.md で毎回 1 から自動生成(履歴はファイルとして蓄積)
- ・fail 時のトリアージは 3 分類。(a) だけ AI が自走し、(b)(c) は人の承認が要る:

表 6-1

分類	意味	対応
(a) 実装バグ	実装が①を満たしていない	AI が④修正 → ⑤再実行を自走(上限 3 回)
(b) テストコードバグ	③が②とズレている	ISSUE の証拠を人が承認 → ③再生成 → 再 freeze
(c) 仕様が誤り	②①自体が間違い	ISSUE で人が裁定 → ①改訂(下流 stale は自動検知)

- ・④⑤ループ上限は 3 回(attempt = 最後の裁定以降の⑤実行回数)。到達で triage ISSUE 自動起票 → ledger に blocked_by 記録 → WBS 🚫。④⑤ skill は blocked 中の実行を拒否
- ・復帰: 人が ISSUE を裁定 → AI が反映(applied) → ledger unblock → ⑤を再トリガ(attempt リセット)。**ブラウザではこれが 1 操作に統合されている** — 裁定待ちの関数の仕様書ページに ISSUE の質問がそのまま出て、回答を送ると「ISSUE への記入(answered)→ unblock → サイト更新」まで済む。リロード後に「⑤を実行する」ボタンに切り替わる

7 ⑥ 完了検証

- ・全関数完了後に 1 回。ledger check(全関数の status・ハッシュ・成果物の総点検)+ review_checks.py all(①②の全関数機械レビュー)が両方 exit 0 で pass
- ・docs/completion-check.md を自動生成。不備リストが出たら該当フェーズへ差し戻し
- ・最終レンダリング(HTML サイト+PDF 合本)もここで実施
- ・LLM を使わないので数十秒で終わる。WBS トップのボタンからも実行できるが、**全関数の⑤が完了するまでボタン自体を出さない**(時期の来ていない操作を見せない)

8 ⑦ 分析・改善

移植完了後の DevOps ループ。1 施策 = 1 票 = 1 イタレーション。

1. **計測・走査:** ここは **2 段階** にしてある。「計測せずに提案しない」を仕組みで担保するため。
 - 1 段階(LLM 不使用): `quant_analyze.py` が `cProfile + radon / ruff / bandit / pip-audit` を機械実行し、実測データを `.legacy-reverse/quant.json` と `docs/quant.md`・`docs/perf.md`に残す
 - 2 段階: AI がその実測データだけを読んで `docs/analysis.md`(OPT- / REF- / SEC- 施策台帳)に候補を記入する。**施策の適用・コード変更・施策票の起票はここではしない**

ホットスポット判断は**代表ワークロード**(ルートの `bench.py`)で行う。無い場合はテスト負荷のスモーク計測になり、その旨が計測サマリに明記される
 2. **項目確定:** 候補を施策票(`docs/improvements/OPT-001.md` 等)に昇格。票には目的・**検証可能な成功基準(baseline → 目標)**・改善仕様。人が承認
 3. **イタレーション:** 適用(1 施策 = 1 コミット)→ **テスト全 pass 確認** → 再計測 → 実測と判定を記録。達成なら **achieved** 、未達なら巻き戻して振り返り
 4. WBS の「**⑦改善イタレーション**」表で全施策の達成状況をトレース
- **挙動保存が絶対条件:** ③のテスト資産を安全網に「テスト全 pass 維持」で適用。挙動変更が必要なら ISSUE → ①②経由(⑦では行わない)。tests/ の編集も禁止
 - Rust(PyO3) 化のような大施策は 4 段階基準(CPU バウンドか → NumPy で足りないか → 純粋関数か → 保守コスト明記)で根拠付き提案 → 人が票で判断
 - status 遷移: `proposed` → `approved` → `applied` → `verified`(`achieved` / `not-achieved`)